

Chordly

Mehr Anand (Project Manager), Mihalis Koutouvos, William Sartorio, Nalini Singh

1. Motivation & Problem Statement

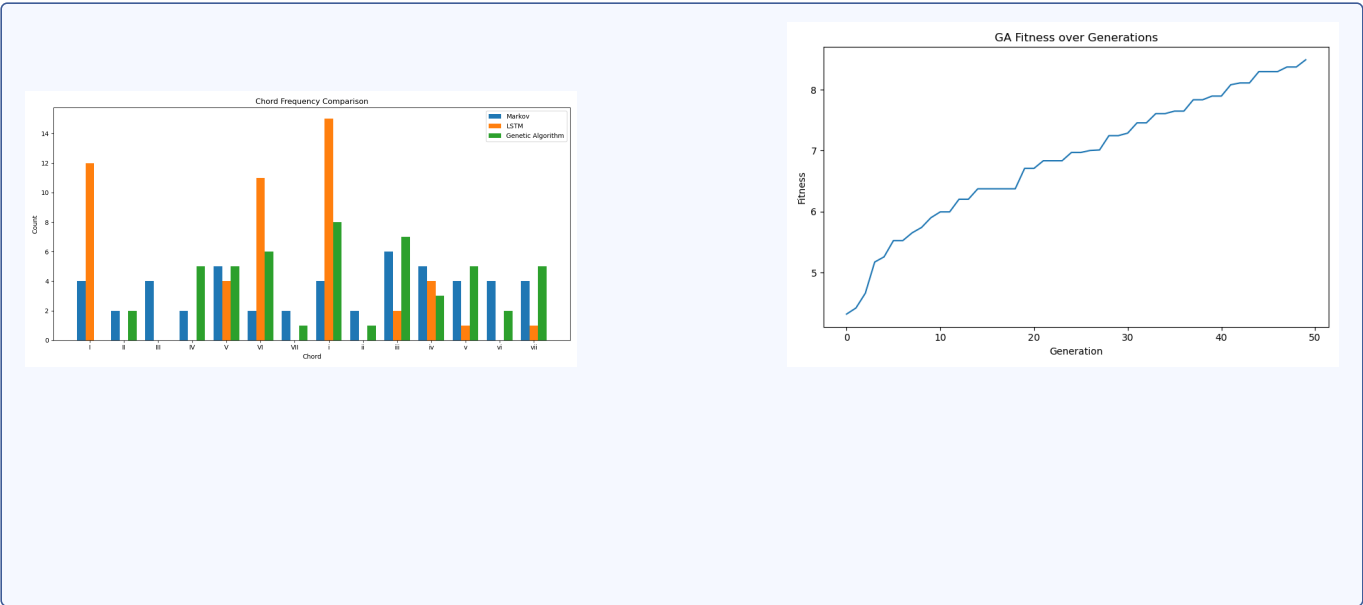
Traditional musical chord progression involves following complex patterns that blend statistical regularities (such as transitions) with theoretical constraints (such as where the most logical and best-sounding note should go, given a progression). With three AI models for generating coherent chord progressions based on data and learning patterns, we aim to be able to simplify and investigate where this problem can be ameliorated. In *Harmonising Chorales by Probabilistic Inference* by Allan and Williams, the two train a Hidden Markov model on a dataset of harmonizations to try to produce new harmonies following previous snippets (Allan and Williams, 2004). Our team wanted to expand in another direction related to these studies to see how well models could propagate sheets of music with right-sounding notes to make wonderful music. With that said, another source of our motivation toward this problem stems from our collective admiration for music and the desire to aid in supporting traditional music composition by having AI models capture the complexity behind musical constraints. Moreover, specifically within the music sector, specific studies related to chord progressions have been minimal, as we discussed earlier, so our group wanted to take that opportunity to develop some kind of a solution or suggestion for future studies in music involving AI utility and more. Lastly, we endeavored to make this project as big of a learning experience as possible, implementing the models by hand, leveraging TAs, and learning lots of new material related to music and coding in general.

2. Approach

Our system investigates two paradigms for algorithmic chord progression generation: statistical learning from data versus rule-based evolutionary optimization. All three models operate over a shared vocabulary of fourteen diatonic Roman numeral chords (I–VII, major and minor), derived from a dataset of chord progressions independently compiled by a team member prior to the course. The Markov chain model constructs a first-order transition probability matrix based on the data, sampling the next chord at each step based on observed bigram frequencies. The LSTM model, built using PyTorch, learns longer-range harmonic dependencies through a two-layer recurrent architecture with an embedding layer, dropout regularization, and a fully-connected output head trained via cross-entropy loss and the Adam optimizer. The Genetic algorithm takes a data-free generative approach, evolving a population of random progressions over fifty generations using single-point crossover and random mutation, with fitness scored by the sum of corpus-derived bigram transition probabilities. One of the main design decisions was to separate the generative models from the audio rendering pipeline: model outputs are Roman numeral sequences, which are converted to MIDI using the music21 library and rendered to WAV audio using FluidSynth with the GeneralUser GS soundfont. The backend is implemented as a FastAPI application exposing three endpoints – /midi, /audio, and /chords – and the frontend is built with Streamlit to provide interactive generation and side-by-side model comparison. We assume all progressions are in a major or minor diatonic context and restrict the chord vocabulary to the seven standard scale degrees, excluding extended or chromatic chords present in the dataset. A scope limitation is that the system does not model rhythm, dynamics, or voice leading explicitly as the output quality is evaluated purely at the level of harmonic sequence. This architecture helped cleanly decouple model logic, audio synthesis, and user interface in order to make it straightforward to swap in additional models or evaluation metrics in future work.

3. Key Results

The results of the models we worked on proved to be somewhat expected but also surprising. To begin, our models and visualization clearly outlined their outputs in a readable way, and the chord progressions made proved to be very well designed with a few inconsistencies in sounds (these are discussed in the Challenges and Lesson Learned section). One chart we are very happy with is the “GA Fitness over Generations” chart (Figure 2; to the right), where we list out the fitness of the chords made over time where the chords are divided into generations and we track how fit each it; as indicated by the gradual upward trend, we can see that the model learned to pull the best features from past iterations of chord generations to produce even better sounding ones. Another useful graph we found is “Chord Frequency Comparison” (Figure 1; to the left): This graph visualized each chord and listed out how common each chord was within the given run we did for the visual; this result may vary, but across several runs the results were consistent with the graph below. As we can see, chords I and i appear the most, especially in the LSTM and genetic algorithm models; the Markov model, however, has results that are pretty even throughout, so most chords in that model appear at a fairly even rate. Another graph not listed is the “Model Metrics Comparison,” which displays how diverse and repetitive the models are, as well as how well chords transition between other chords. We can take away that the Markov model produces the most diverse set of chords, LSTM is way less diverse, and the genetic algorithm falls in the middle; LSTM is also the most repetitive, while Markov is not; however, transition validity holds the same across each model. With that said, as a group, we definitely thought the genetic algorithm would produce the most diverse results, but it turned out to be the Markov model. In fact, the genetic algorithm proved to be the most average out of all the models, while LSTM was not ideal in most cases due to how repetitive it was.



4. Challenges & Lessons Learned

One of the primary technical challenges was the cross-platform compatibility of FluidSynth, the library used for MIDI-to-audio rendering. While installation on macOS was straightforward via Homebrew, setting up FluidSynth on Windows required manually downloading binaries and configuring system PATH variables, which proved time-consuming and error-prone. This limited the number of team members who could run the full audio pipeline locally, with the complete system only functioning on a single laptop for a significant portion of development. As a partial mitigation, the audio rendering logic was encapsulated within a dedicated backend module so that future deployment to a shared server would allow team members to access the system through the frontend without requiring a local FluidSynth installation. A second significant challenge was model evaluation. Unlike classification or regression tasks, there is no straightforward quantitative metric for musical quality, so the primary evaluation criterion is ultimately reduced to subjective listening, which is inconsistent across listeners. We attempted to supplement this with structural metrics such as unique chord count and major-to-minor chord ratio, but these do not fully capture harmonic coherence or aesthetic appeal. A lesson learned is that future work in generative music should establish evaluation criteria early, potentially incorporating user studies or established music information retrieval metrics such as chord entropy or transition perplexity.

5. Individual Contributions

Member	Primary Contributions	Approx. %
Mehr Anand (PM)	System Design, frontend development, slides, aided with Markov and LSTM	25%
Mihalis Koutouvos	Markov model, model visualization scripts, reports, proposals	25%
William Sartorio	Genetic algorithm, LSTM model, endpoints for frontend	25%
Nalini Singh	Frontend development, audio mechanic	25%

6. Repository & References

GitHub Repository: <https://github.com/Sadfahlsdj/CS4100-Project/tree/main>
Ensure repo is public, or add instructor as a collaborator.

Project Tracker: <https://github.com/Sadfahlsdj/CS4100-Project/activity>

Key References:

- [1] ANDREW HUANG, “Learn music theory in half an hour.,” YouTube, 2020: Video. This resource provided an accessible overview of Music Theory, providing the foundational background for teammates unfamiliar with these concepts.
- [2] Michael New, “How the Roman Numeral System Works - Music Theory,” YouTube, 2013: Video. This resource provided an accessible overview of Roman Numeral System in Music Theory, providing the foundational background for teammates unfamiliar with these concepts.
- [3] Moray Allan and Christopher K. I. Williams, “Harmonising Chorales by Probabilistic Inference,” NeurIPS, 2004. This paper describes how their team used a dataset of chorale harmonisations from Bach to train Hidden Markov Models